Now I'd like to show how you can use TDD to move logic outside the Rails classes. The workflow class needs to create a project from a name and a list of tasks. Let's start with the name:

test_first/01/spec/workflows/creates_project_spec.rb

```
require "rails_helper"

RSpec.describe CreatesProject do
  it "creates a project given a name" do
    creator = CreatesProject.new(name: "Project Runway")
    creator.build
    expect(creator.project.name).to eq("Project Runway")
  end
end
```

It's a straightforward test, which is the point. Because the logic isn't in the controller, you don't need to do anything fancy to test it. I'm calling the class CreatesProject because I like having action classes that aren't nouns. Alternate naming conventions might include CreateProject, ProjectCreator, or ProjectFactory.

And the passing code looks like this:

test_first/01/app/workflows/creates_project.rb

```
class CreatesProject
  attr_accessor :name, :project

  def initialize(name: "")
    @name = name
  end

  def build
    self.project = Project.new(name: name)
  end
end
```

One thing to notice about the build method is that it creates the project object but does not save it. Often, when I create an action object I try to separate initialization, execution, and persisting the result. One reason to separate these features is that it is much easier to write fast tests of using the workflow object if I can do so without hitting the database. In addition to making testing easier, this comes in handy when I want to create an object and not save the result immediately, which happens more often than you might think.

Here I'm using Ruby keyword arguments in the initializer, as a cheap type check to make sure the arguments passed to the CreatesProject initializer are limited to the ones you want.

Next the string-parsing features are tested. If you're following along, you should write these one at a time, making each test pass before writing the next: